

Taller: Introducción a React Native

Entornos de desarrollo, navegación entre pantallas y configuración base de Firebase

Duración: 90 minutos | **Modalidad:** individual | **Entrega:** enlace del repositorio en Virtual TdeA

Objetivo. Comprender qué exige el entorno de desarrollo móvil según el sistema operativo, distinguir React Native de React para web, manejar la navegación entre pantallas y dejar configurada la conexión con Firebase en una aplicación funcional.

Requisitos previos. Node.js LTS instalado, un editor de código, un dispositivo físico con Expo Go o un emulador configurado, y una cuenta de Google para la consola de Firebase. La documentación oficial está permitida y es la fuente recomendada durante todo el taller.

Carácter individual. Este taller se desarrolla y se entrega de forma individual. Puede consultar con sus compañeros y con el docente para resolver dudas puntuales, pero el código y las respuestas deben ser de su propia autoría. Se revisará la similitud entre las entregas y el historial de commits del repositorio.

Momento	Tiempo	Peso	Contenido
Parte 1 — Preguntas 1 a 4	30 min	40%	Entornos, fundamentos, navegación y Firebase
Parte 2 — Pregunta 5	55 min	60%	Aplicación de tres pantallas conectada a Firestore
Cierre	5 min	—	Verificación, subida a GitHub y entrega

Parte 1 — Componente teórico

40% · 30 minutos

Responda con sus propias palabras. Máximo 8 líneas por literal. Las respuestas van en un archivo `RESPUESTAS.md` dentro del repositorio.

PREGUNTA 1

Entorno de desarrollo y sistema operativo

a) Complete la tabla explicando qué papel cumple cada pieza del entorno de desarrollo móvil:

Herramienta	¿Qué papel cumple?
Node.js y npm	
Metro bundler	
JDK y Android SDK	
Xcode	
Expo Go	

b) Un estudiante con **Windows** y otro con **Linux** quieren compilar la misma aplicación para **iOS**. Explique por qué ninguno de los dos puede hacerlo en su propia máquina y mencione dos alternativas reales para que igual logren probar y publicar la app.

c) ¿Qué son las **variables de entorno** del sistema operativo y por qué el emulador de Android falla cuando `ANDROID_HOME` o el `PATH` están mal configurados? Explique la diferencia entre una variable de usuario y una variable de sistema.

d) Compare el flujo de trabajo con **Expo** frente a **React Native CLI**: mencione dos ventajas y dos limitaciones de cada uno, e indique en qué situación elegiría cada opción.

PREGUNTA 2

Fundamentos de React Native

a) React Native no renderiza HTML. Complete la equivalencia entre el elemento web y el componente que cumple esa función en React Native:

En la web	En React Native
<code><div></code>	
<code><p></code> o <code></code>	
<code></code>	
<code><input></code>	
Lista larga con scroll	

b) Explique tres diferencias entre los estilos de React Native y el CSS tradicional. Incluya obligatoriamente el valor por defecto de `flexDirection` y por qué esa decisión tiene sentido en un dispositivo móvil.

c) ¿Cuál es la diferencia entre **props** y **estado**? Dé un ejemplo de un dato que debería ser prop y uno que debería ser estado dentro de una pantalla de lista de productos.

PREGUNTA 3

Manejo de pantallas y navegación

a) Describa para qué sirve cada tipo de navegador y proponga un caso de uso real para cada uno:

Navegador	¿Para qué sirve?	Caso de uso
Stack		
Tabs		
Drawer		

- b) ¿Qué función cumple el contenedor que envuelve toda la navegación de la app y por qué debe existir uno solo en todo el proyecto?
- c) Explique cómo se **envía un parámetro** de una pantalla a otra al navegar y cómo lo **lee** la pantalla de destino. ¿Qué tipo de dato conviene enviar: el objeto completo o únicamente su identificador? Justifique.
- d) Cuando el usuario navega de la pantalla A a la B y luego regresa, ¿la pantalla A se vuelve a crear desde cero o conserva su estado? Explique qué implicación tiene esa respuesta cuando la pantalla A muestra una lista que debe refrescarse tras crear un registro nuevo.
- e) ¿Qué diferencia hay en el comportamiento del *regresar* entre Android e iOS a nivel de sistema operativo, y cómo lo resuelve la librería de navegación?

PREGUNTA 4 Configuración base de Firebase

- a) Enumere en orden los pasos necesarios para dejar un proyecto de Firebase listo para ser consumido desde una app de React Native, desde la creación del proyecto en la consola hasta la inicialización dentro del código.
- b) Explique qué identifica cada una de estas claves del objeto de configuración:

Clave	¿Qué identifica?
apiKey	
projectId	
appId	
storageBucket	

- c) La `apiKey` de Firebase queda visible en el paquete de la aplicación y cualquiera puede extraerla. Explique **por qué eso no constituye por sí solo una falla de seguridad** y dónde reside entonces la protección real de los datos.
- d) ¿Qué diferencia hay entre iniciar Firestore en **modo de prueba** y en **modo de producción**? ¿Qué riesgo concreto corre una app que se publica con las reglas en modo de prueba?
- e) Compare **Cloud Firestore** y **Realtime Database** en al menos tres aspectos, e indique cuál elegiría para la aplicación de la Parte 2 y por qué.

Parte 2 — Componente práctico

60% · 55 minutos

PREGUNTA 5 Aplicación "Directorio de Contactos"

EL RETO

Construya una aplicación móvil de **tres pantallas** con navegación tipo Stack, conectada a **Cloud Firestore**, que permita listar contactos, ver el detalle de uno y registrar uno nuevo. El taller indica *qué* debe lograr y *qué* debe verse; el *cómo* es su trabajo. Está permitido consultar la documentación oficial, no soluciones completas de terceros.

PASO 1 Proyecto y entorno 8 min

- Cree un proyecto nuevo de React Native con Expo e instale las dependencias de navegación.
- Ejecútelo en un dispositivo físico con Expo Go o en un emulador.
- Modifique el texto de la pantalla inicial y confirme que el cambio se refleja sin reiniciar la aplicación.

RESULTADO ESPERADO

La aplicación abre en el dispositivo mostrando su texto personalizado. Al guardar un cambio en el editor, la pantalla se actualiza sola en menos de dos segundos. En la terminal aparece el bundler corriendo sin errores en rojo.

PASO 2 Navegación entre las tres pantallas 12 min

Configure un navegador tipo Stack con las siguientes pantallas:

Pantalla	Contenido	Navegación
Lista	Listado de contactos y un botón para agregar uno nuevo.	Es la pantalla inicial. Lleva a <i>Detalle</i> al tocar un contacto y a <i>Nuevo</i> con el botón.
Detalle	Nombre, teléfono y ciudad del contacto seleccionado.	Recibe el identificador del contacto como parámetro. Permite regresar a <i>Lista</i> .
Nuevo	Formulario con tres campos de texto y un botón de guardar.	Al guardar, regresa automáticamente a <i>Lista</i> .

El encabezado de cada pantalla debe mostrar un título en español, y en la pantalla de detalle ese título debe corresponder al nombre del contacto abierto.

RESULTADO ESPERADO

Puede recorrer las tres pantallas con datos quemados en el código, sin Firebase todavía. El botón físico de retroceso de Android y el gesto de deslizar en iOS regresan correctamente. El título del encabezado cambia según la pantalla.

PASO 3

Configuración de Firebase

12 min

- Cree un proyecto en la consola de Firebase y registre dentro de él una aplicación.
- Habilite **Cloud Firestore** y cree una colección llamada `contactos`.
- Agregue manualmente **tres documentos** de prueba desde la consola, cada uno con los campos nombre, teléfono y ciudad.
- Instale el SDK de Firebase en el proyecto y centralice la inicialización en un **único archivo** de configuración que el resto de la app importe.
- Los valores de configuración no deben quedar escritos directamente en el código versionado.

RESULTADO ESPERADO

La aplicación arranca sin errores de inicialización de Firebase y en la consola de Firestore se ven los tres documentos creados dentro de la colección `contactos`.

ADVERTENCIA

Si sube al repositorio el archivo con sus credenciales de Firebase, el criterio de seguridad de la rúbrica se califica en cero. Use un archivo de variables de entorno excluido por `.gitignore` y publique en su lugar un archivo de ejemplo con los nombres de las variables y valores vacíos.

PASO 4

Lectura y escritura de datos

18 min

- La pantalla **Lista** obtiene los contactos desde Firestore y los muestra en una lista con desplazamiento eficiente, no en un contenedor de scroll simple.
- Mientras los datos cargan se muestra un indicador de carga, y si la colección está vacía se muestra un mensaje que lo indique.
- La pantalla **Detalle** muestra los datos del contacto que corresponde al identificador recibido como parámetro.
- La pantalla **Nuevo** guarda un documento en Firestore con los tres campos y no permite guardar si algún campo está vacío.
- Tras guardar, la pantalla **Lista** debe reflejar el contacto nuevo sin necesidad de cerrar y volver a abrir la aplicación.

RESULTADO ESPERADO

Al abrir la app aparecen los tres contactos creados desde la consola. Al registrar uno nuevo desde el formulario, este aparece en la lista al regresar y también queda visible en la consola de Firebase. Al intentar guardar con un campo vacío, la aplicación lo impide y avisa al usuario.

PASO 5

Verificación final

5 min

- ☐ La app corre en dispositivo o emulador sin pantallas rojas de error.
- ☐ Las tres pantallas navegan en ambos sentidos y los títulos son correctos.
- ☐ El detalle recibe y usa el parámetro enviado desde la lista.
- ☐ Los datos provienen de Firestore, no de un arreglo quemado en el código.
- ☐ Un contacto creado desde la app aparece en la consola de Firebase.
- ☐ Las credenciales no están versionadas en el repositorio.

Retos opcionales

bono de 0.5

1. Reemplace la consulta única por una **suscripción en tiempo real**: un cambio hecho desde la consola de Firebase se refleja en la app sin tocar nada.
2. Agregue la opción de **eliminar** un contacto desde la pantalla de detalle, con confirmación previa.
3. Cambie el navegador Stack por una combinación de **Tabs** con dos secciones (Contactos y Acerca de), manteniendo la navegación de detalle dentro de la primera.
4. Ajuste las **reglas de seguridad** de Firestore para que la colección sea de lectura pública pero de escritura restringida, y documente la regla en el `RESPUESTAS.md`.

Entregable

Se entrega un único enlace: la URL del repositorio público en GitHub, publicada en la actividad correspondiente de Virtual TdeA.

El repositorio debe contener:

- El código completo de la aplicación, sin la carpeta `node_modules`.
- Un archivo `RESPUESTAS.md` con las respuestas de las preguntas 1 a 4.
- Un archivo `README.md` con su nombre completo, el sistema operativo en el que trabajó y los pasos para ejecutar el proyecto.
- Un archivo de ejemplo con los nombres de las variables de entorno de Firebase, sin valores.
- Un `.gitignore` que excluya `node_modules` y el archivo de credenciales.
- Una carpeta `capturas/` con cuatro imágenes: las tres pantallas de la app y la consola de Firestore mostrando el contacto creado desde el dispositivo.

Los repositorios privados o los enlaces que no abran no se califican. Verifique la URL en una ventana de incógnito antes de publicarla en Virtual TdeA.

Rúbrica de evaluación

Criterio	Peso
Preguntas 1 a 4: claridad, corrección conceptual y justificación de las respuestas	40%
El proyecto ejecuta sin errores en dispositivo o emulador	10%
Navegación: tres pantallas, paso de parámetros y títulos correctos	20%
Firebase: inicialización centralizada, lectura y escritura funcionando	20%
Manejo seguro de credenciales y repositorio completo y ordenado	10%
Total	100%

Documentación recomendada: la guía de inicio de Expo, la sección *Fundamentals* de React Navigation y la guía *Get started with Cloud Firestore* en la documentación oficial de Firebase.